The Watcher That Moves

Policies a Robot Can Follow, and Secrets Kept
from a Watcher That Knows the Plan

Haniel Ulises

Epistemic Robotics

14 October 2026

The watcher of the previous report stood still and reasoned only from what it saw. Here it may change posts, out of everyone's sight, while the carrier drives, and it may know the policy. Three results. First, a planner that branches on every designated event returns, for a watcher that moves, policies no robot can follow: after a drive they branch on whether the watcher moved, which the carrier cannot see. Branching instead on what each action's actor observes, a change to Aletheia, the planner finds the carrier looking down the hall before it tells and changing windows when the watcher has come; on every other domain the change leaves the policies as they were. Second, a policy that is correct under the product update can still give the order away to a watcher that knows the policy: the shortest one, which uses the gates, does, for every number of moves we tried, and the one without the gates does not. Third, in Gazebo, over 20 seeds under five formulations and two watcher strategies, judged by the robots' own lasers and by a watcher that knows the policy from the other runs, the policy without the gates kept the order in 20 of 20 runs against the watcher's best strategy; the published policy kept it in 0, the shortest in 0, and the classical carriers in 0 and 10.

Contents

1 Scope **2**

2 The hall and the domain **2**

3 Policies a robot can follow **2**

4 A secret from a watcher that knows the policy **3**

5 Paired evaluation over 20 seeds **4**

6 Recorded runs **5**

7 Changes to the executor and to the hall **5**

8 Limitations **5**

1 Scope

The previous report ended on a limitation: the watcher did not move and did not reason about the policy. Both matter. A watcher that moves makes what the carrier read about it out of date by the time the carrier acts, and a planner has to account for that without pretending the carrier can see what it cannot. A watcher that knows the policy reads the carrier's choices, not only its signals, and the product update, which says what an agent learns from the events it observes [1], does not say what such an agent learns.

2 The hall and the domain

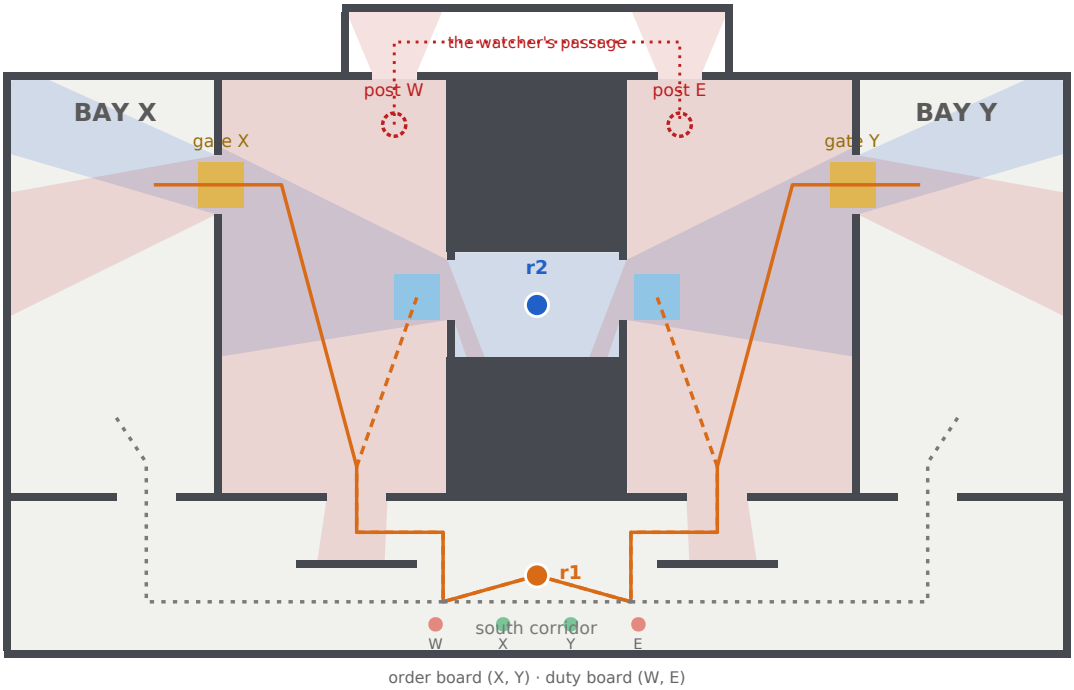


Figure 1: The hall of the previous report, with the posts nearer the block and a passage behind the north wall joining them, dotted red. Shaded as before: blue, what the ally sees; red, what a watcher sees from each post.

The watcher changes posts through the passage in 6.0 m, against 6.6 m for the carrier's shortest drive to a window, so a watcher that leaves when the carrier starts to drive is at its new post before the carrier arrives (in the runs it was, in 60 of 60 moves).

The domain, written by `make_domain.py` for K changes of post, has the carrier at one of five places, its start, the two windows and the two gate approaches, and every drive between them is an action with one event per thing the watcher may do while it lasts: stay, or go to the other post if it has a change left. Only the watcher tells them apart. The carrier can look down the hall it stands in, from a window or a gate approach. Observability follows the sightlines, except that the watcher is taken to see every drive, which is cautious. The reading of the boards is semi-private: everyone knows the carrier reads them. The goal is the previous report's, $(\mathit{in}\text{-}x \vee \mathit{in}\text{-}y) \wedge Kw_{r2}\ \mathit{dest}\text{-}x \wedge \neg Kw_{r3}\ \mathit{dest}\text{-}x$, and every plan is made with `-consistent-beliefs`, which refuses states in which an agent believes a contradiction.

3 Policies a robot can follow

AO* over a multi-pointed model branches after an action on its designated events, one branch each. That is right for a sensing action whose outcome the actor observes, and wrong for one whose

outcome it does not: after a drive, Aletheia's policy branches on whether the watcher stayed or moved. The carrier cannot see that, and so cannot know which branch it is on. Such a policy is correct for an executor that is told the world, and no robot executes it.

Definition 1 (Subjective branching)**.** After action a by actor i in state s, the branches are the classes of the designated worlds of $s \otimes a$ under the transitive closure of i's accessibility restricted to them; each branch designates its class.

A branch is then something the actor can know it is on. When i observes every event of a fully, the classes are the events and nothing changes. The change to Aletheia (branch `feature/subjective-branching`) merges the per-event branches of `product_update_split` by the actor's relation; `-subjective auto` takes the actor of a ground action to be the first agent its name gives after the schema, the convention of every domain written for ePlanSys. Without the option the planner returned the same expansions and validity as the installed one on 28 of its 28 benchmarks. With it, on 24 of the 24 tasks of the EPDDL workspace's other domains, every policy has the depth and the leaves it had before (`tools/regression.sh`); only the watcher that moves changes. `tools/executable.py` counts, in a policy, the branch points whose events the actor cannot tell apart (table 1).

		every designated event					**what the actor observes**					
	K	depth	nodes	expanded	s	**unfollowable**	depth	nodes	expanded	s	unfollowable	**opaque**
with gates	0	5	13	631	0.11	**0**	5	13	631	0.12	0	**no**
with gates	1	6	27	1 431	0.75	**8**	7	19	4 120	2.24	0	**no**
with gates	2	7	39	6 497	5.00	**12**	9	27	54 230	33.43	0	**no**
with gates	3	8	47	19 279	55.18	**16**	11	33	523 158	283.40	0	**no**
no gates	0	5	15	205	0.02	**0**	5	15	205	0.03	0	**yes**
no gates	1	6	27	407	0.07	**2**	7	31	560	0.09	0	**yes**
no gates	2	7	39	830	0.20	**4**	9	47	1 500	0.29	0	**yes**
no gates	3	8	51	1 539	0.34	**6**	11	63	3 740	1.02	0	**yes**

Table 1: The planning study (`tools/study.py`), with the gates and without them, for K changes of post. Unfollowable: branch points at which the actor cannot tell the branches' events apart. Opaque: whether the policy keeps the order from a watcher that knows it, by the sightlines (section 4); for a policy no robot can follow the question does not arise.

Under subjective branching the policy is longer, depth $2K + 5$ against $K + 5$, and the difference is the point: one look per change of post the watcher may have made. The policy for $K = 1$, without the gates, reads the order and the duty board, drives to the window on the side the watcher is not, and looks down that hall. If the hall is clear it shows the lamp and goes in by the back route. If the watcher has come, it drives to the other window and shows the lamp there without looking again: the watcher has used its change of post.

4 A secret from a watcher that knows the policy

A policy is *opaque* to the watcher if any two worlds the watcher cannot tell apart at the start, and that differ on the order, produce the same set of traces of what the watcher sees, over every choice the watcher makes. Then whatever it sees, and whatever it did, a world of either order explains it. `tools/opacity_sightlines.py` computes the traces from the hall, not from the domain's declarations: after each step the watcher knows what it did, sees the carrier only on its own side, and the lamp and the gate only of its own side.

The shortest policy is not opaque, for any K we tried. With $K = 0$ it does not even read the duty board: it drives to the window on the side opposite its bay and looks; if the watcher is there it crosses to its bay's gate, which it now knows is unwatched. Under the product update that is sound, since a carrier at a window says nothing about the bay. A watcher that knows the policy reads it as it is: the carrier came to my window, so its bay is on the other side. Without the gates

the carrier's visible choices depend on where the watcher is, never on the order, and the policy is opaque for every K of table 1.

That opacity is a property the planner does not search for. It was found here by removing the gates from the domain and checking the result; `tools/plan_opaque.py`, which removes the action schema at which a counterexample first shows the watcher something and plans again, removes a window drive or a gate approach and is soon left with no policy: what leaks is not an action but when it is chosen.

5 Paired evaluation over 20 seeds

Five formulations: the published policy (*v1*), the shortest policy with the gates (*shortest*), the opaque one (*opaque*), both planned with $K = 1$ and subjective branching, and the classical *direct* and *discreet* carriers of the previous report. Two watcher strategies: *stay*, and *chase*, which, knowing that a carrier that reads the duty board tells on the side the watcher is not, leaves for the other post the moment the carrier starts its first drive. Every seed is run under all ten pairs from the same start.

The watcher is judged twice. *By the events* it names a bay if its laser had the carrier in a gate or a bay, or saw the lamp, as in the previous report. *By the policy* it has also watched the policy run before: it names the bay of every other run of the same formulation and strategy, from the same post, that showed it the same trace of places and lamp, if they all had one bay. A run *hides* the order when the carrier delivered, the ally named the bay, and the watcher named none either way.

formulation	watcher	hidden	ally named	named by events	named by the policy	reported success	median
published (v1)	stay	20/20 [0.84, 1.00]	20/20	0/20	0/20	20/20	127.2 s
published (v1)	chase	0/20 [0.00, 0.16]	20/20	20/20	20/20	20/20	126.7 s
shortest, gates	stay	0/20 [0.00, 0.16]	20/20	0/20	20/20	20/20	189.3 s
shortest, gates	chase	0/20 [0.00, 0.16]	18/20	0/20	20/20	18/20	84.2 s
opaque, no gates	stay	20/20 [0.84, 1.00]	20/20	0/20	0/20	20/20	173.1 s
opaque, no gates	chase	20/20 [0.84, 1.00]	20/20	0/20	0/20	20/20	263.1 s
direct classical	stay	0/20 [0.00, 0.16]	20/20	10/20	20/20	20/20	78.5 s
direct classical	chase	0/20 [0.00, 0.16]	20/20	10/20	20/20	20/20	77.8 s
discreet classical	stay	10/20 [0.30, 0.70]	20/20	10/20	10/20	20/20	164.4 s
discreet classical	chase	10/20 [0.30, 0.70]	20/20	10/20	10/20	20/20	163.8 s

Table 2: The batch, 20 runs per cell. Hidden, with the Wilson 95% interval. Named by the policy includes named by the events. Median: simulated seconds from the common start, over runs the executor reported as a success.

Against its watcher's better strategy each formulation kept the order in: published (v1), 0 of 20 (against chase); shortest, gates, 0 of 20 (against stay); opaque, no gates, 20 of 20 (against stay); direct classical, 0 of 20 (against stay); discreet classical, 10 of 20 (against stay). The only runs that did not end in success are 2 of the shortest policy against the chasing watcher, each at a second look whose request the executor sent and no node answered; they count against delivery, not against the secret.

6 Recorded runs

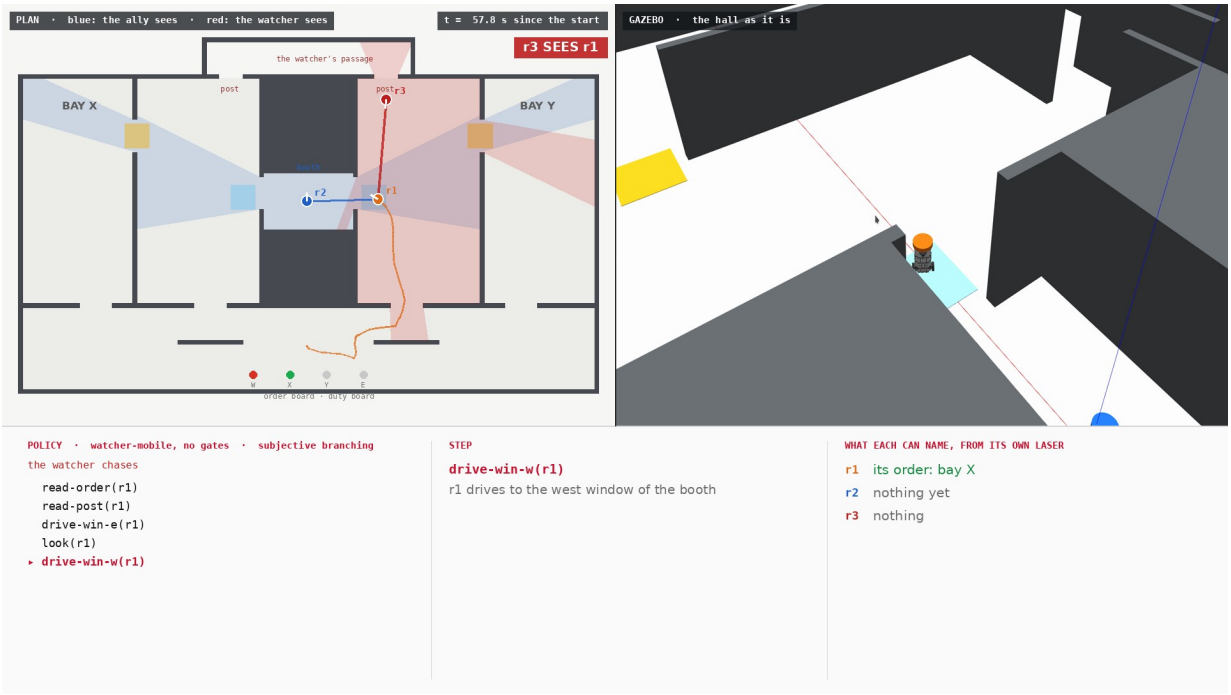


Figure 2: A frame of the film: the opaque policy against the chasing watcher. The carrier, at the east window, has looked down the hall and found the watcher, who came through the passage while it drove; it will cross to the west window. Left, the hall from above, the watcher's sight shaded from where it stands; right, Gazebo; below, the actions run, the step, and what each robot can name.

The film: Seed 1, where the order names bay X and the watcher starts at the west post: published (v1) against a watcher that chases; shortest, gates against a watcher that stays; discreet classical against a watcher that chases; opaque, no gates against a watcher that chases.

7 Changes to the executor and to the hall

ePlanSys's executor keeps its own model of the mission and applies each action's outcome to it. A drive whose outcomes the carrier cannot tell apart has no outcome to report, and the executor asked for one. Its state node and the planner plugin, built over the changed Aletheia, take a parameter for subjective branching and keep the unseen outcome as uncertainty. They are built in an overlay workspace over the installed ePlanSys, which they do not replace.

The baffles that hide the corridor from the posts left a pocket 0.6 m tall between each baffle and the corridor wall, where the carrier's controller sometimes oscillated: seven runs of the previous report stalled there, and here, where crossing between the windows goes through both pockets, the first runs did too. The baffles were lowered to leave 0.85 m, the views checked again (`tools/check_views.py`), and the batch begun again; the 20 runs of the first, 1 of them stalled, are kept apart as discarded.

The sightlines variant of the previous report hid the boards' reading from the ally. The ally then held the lamp impossible and believed a contradiction, and its knowledge held vacuously. That variant was corrected and republished; every plan here is made with `-consistent-beliefs`.

8 Limitations

The runs give the watcher one change of post; the planning study goes to $K = 3$. The domain assumes that the watcher moves only while the carrier drives and reaches its new post first, and the

hall is built so that it does: in the runs it arrived first in 60 of 60 moves. Opacity is checked rather than planned for: the opaque policy comes from a domain without the gates, and a planner that searched for opacity would have to compare the runs of two worlds as it searched. The watcher that knows the policy learns it from the other runs of the batch, five per world and strategy.

References

[1] A. Baltag, L. S. Moss and S. Solecki. The logic of public announcements, common knowledge, and private suspicions. *Proceedings of TARK VII*, 43–56, 1998.

[2] T. Bolander and M. B. Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics* 21(1):9–34, 2011.

[3] A. J. Coles, A. I. Coles, M. Fox and D. Long. Forward-chaining partial-order planning. *Proceedings of ICAPS*, 42–49, 2010.